

A Two-Stage Distributed Framework for Flight Delay and Severity Prediction Using Spark MLlib

ST446: Distributed Computing for Big Data — Final Group Project

Candidates 70868, 70624, 60602, & 66583

London School of Economics
London, United Kingdom

Abstract

This paper addresses two interconnected flight delay prediction tasks: binary classification of whether a flight is delayed, and severity classification of delayed flights as minor (15–45 minutes) or major (>45 minutes). To solve both tasks at scale, we propose a two-stage distributed classification framework built on Apache Spark MLlib, benchmarking four classifiers: Logistic Regression, Random Forest, Gradient Boosted Trees, and a Stacking ensemble, across three major US hub airports (JFK, ATL, ORD) on 3.4 million flight records. Class imbalance in Stage 1 (5.4:1 ratio) is addressed through inverse-frequency class weighting for LR and RF, and downsampling for GBT. GBT achieves the strongest Stage 1 performance (AUC-ROC 0.703, precision 0.786), while Stacking produces the highest F0.5 of 0.780. Stage 2 performance is broadly similar across tree-based models (AUC-ROC 0.624–0.656), with Random Forest delivering comparable accuracy at a fraction of the training time (17.7s vs 132.6s for Stacking), making it the most practical Stage 2 deployment choice.

1 Introduction

1.1 Description

Flight delays are a persistent and costly challenge in modern aviation, with cascading effects on airline operations, passenger satisfaction, and the broader economy [2]. Predicting not only whether a flight will be delayed, but how severely, is therefore of significant practical and economic value. This work addresses both problems through a two-stage prediction framework: Stage 1 predicts whether a flight is delayed, defined by the US Federal Aviation Administration as a departure 15 minutes or more past its scheduled time [6]; Stage 2 predicts the severity of that delay, classifying flights departing 15–45 minutes late as minor delays and those exceeding 45 minutes as major delays [2, 6]. Globally, air travel carried 9.8 billion passengers in 2025 [1], and the Travel & Tourism industry accounts for roughly 10% of global GDP and supports 336 million jobs [13]. Disruptions to air travel, therefore, carry significant social and economic consequences, making accurate delay prediction a valuable tool for airlines seeking to optimise operations, manage crew reallocation, and maintain passenger satisfaction [2].

1.2 Importance

The act of translating practical importance of delay prediction into accurate, scalable models is non-trivial; existing approaches face three interconnected challenges that this work seeks to address. Firstly, the problem itself exhibits significant class imbalance, as most flights arrive on time, which prior work has often failed to

address [2]. Secondly, flight arrival/departure data can be very high dimensional and heterogeneous, as a thorough analysis would require analysis of temporal, operational, and weather-related features to predict delays accurately. Furthermore, to make predictions on different locations in the US for better generalization, we need to work with very large datasets, containing hundreds of thousands of rows, which requires the use of distributed machine learning pipelines, such as Apache Spark, to achieve scalable and efficient model training.

1.3 Our Solution

To overcome the challenges of class imbalance, high dimensionality, and limited generalizability identified in prior work, we propose a two-stage distributed classification framework built on Apache Spark ML pipelines, enabling scalable model training across three major US airports. For each stage we will evaluate four classifiers: a logistic regression baseline, a Random Forest (bagging), a Gradient Boosted Tree classifier (boosting), and a stacking ensemble combining the outputs of the Logistic Regression and Random Forest. While Stage 1 mirrors the binary classification focus of most prior work, our framework extends this by operating at scale across multiple US airports using distributed Spark pipelines. We chose to use a stacking ensemble as it improves robustness by combining the strengths of LR and RF. Class imbalance is addressed in the stacking ensemble through class weighting within each base learner, making it an ideal method to address the class imbalance of our dataset, especially in Layer 1. Furthermore, most existing approaches are limited to binary on-time/delayed classification and rely on single-airport datasets, limiting their generalization [8]; our work addresses this by extending predictions to delay severity across three major US airports (JFK, ATL, ORD) using a scalable distributed pipeline. These models were evaluated based on evaluation/accuracy metrics (F0.5, precision, recall, and AUC), and scalability metrics such as measuring training time using incremental 25% partitions of the data (increasing to 100%).

1.4 Summary of Results

Across both layers, the four models produced broadly similar accuracy scores, with differences of no more than 2% between the LR baseline, RF, and stacking ensemble. This convergence is partly attributable to the class imbalance in Layer 1, where approximately 82% of flights are on time. This imbalance that inflates accuracy for all models regardless of their true predictive power, making recall a more informative metric at this stage. The GBT model achieved the

lowest accuracy on Layer 1 (0.59) but recorded the highest AUC-ROC of any model (0.7032), which suggests stronger discriminative ability that raw accuracy fails to capture. In terms of scalability, LR was the most efficient model with an average training time of 8.2s across layers, compared to 158.8s for RF, and 67.95s for GBT, making LR the most practical choice when training time is a constraint. These results contribute to the literature on flight delay prediction by benchmarking four classifiers across three major US airports (JFK, ATL, ORD), and by demonstrating that a distributed two-stage framework can address class imbalance at scale, which prior work has failed to address. [2, 8].

2 Related Work

Flight delay prediction has been widely studied using machine learning classification models and frequently handles the methodological concern of class imbalance in available data. Thiagarajan et al. [10] establish a layered delay prediction, using Bureau of Transportation Statistics to develop a two-stage model across 15 major US airports where Stage 1 is a binary classification task and Stage 2 performs regression to predict the exact delay in minutes. GBT achieves the best classification accuracy of 94.35% and Extra-Trees the best regression MSE. Class imbalance is addressed via SMOTE combined with Tomek Links, though models are trained on a single machine without a distributed framework.

Further building on the importance of class imbalance handling, Albassam and AlShahrani[2] conduct a thorough evaluation of six classifiers across three resampling strategies meant to counter class imbalance, Random Oversampling, SMOTE, and ADASYN, framing delay prediction as a single three-class problem distinguishing small, medium, and large delays. Their best performing models, Random Forest with Random Oversampling and Decision Tree with SMOTE, achieve an accuracy of 0.90 and F1-score of 0.91 using scikit-learn on a single machine, but do not address scalability to larger datasets.

In contrast, Vonitsanos et al. [11] address this scalability concern directly, applying Apache Spark MLlib to binary flight delay classification on a 4-node distributed cluster. Deploying Random Forest, Multilayer Perceptron, Naive Bayes, and SVM, they found Random Forest to achieve the best F-measure of 63.75%. While they demonstrate the merit and applicability of a distributed framework for flight delay prediction, their evaluation is limited to a single airline, American Airlines, and contains no additional delay classification to categorize severity.

Our work combines these three contributions into a single study that addresses the limitations of each on its own. From Thiagarajan et al. we utilize the two-stage hierarchical structure, but replace the regression second stage with a severity classification task, predicting whether a delay is minor or major to produce a more operationally actionable output for airline use. From Albassam and AlShahrani we adopt systematic class imbalance handling, applying it within a distributed setting that their single-machine approach cannot scale to. From Vonitsanos et al. we similarly deploy the Apache Spark MLlib distributed pipeline, extending it beyond binary classification to a two-layer severity framework across three major US hub airports (JFK, ATL, ORD) on 3.4 million flight records. Limited work combines all three of these elements of layered classification,

methodological imbalance handling, and distributed scalability in a single flight delay prediction framework.

3 Methodology

3.1 Data Acquisition & Preparation

Our raw data consists of monthly U.S. domestic flight records sourced from the Bureau of Transportation Statistics (BTS) On-Time performance dataset [5] across the years 2018 to 2022. Each monthly file was saved as a separate CSV within a single ZIP archive.

We loaded the raw CSVs into Apache Spark, using path matching over the nested directory structure and `inferSchema=true` to resolve column types. The consolidated DataFrame was then serialized into a compressed 4-part parquet for faster data loading in the future. All subsequent stages of our pipeline read directly from these raw parquet files.

The cleaning stage applied four filters meant to prepare the data for modelling. First, scope was restricted to the three target hub airports: Chicago O'Hare (ORD), New York JFK, and Atlanta Hartsfield-Jackson (ATL). Second, we removed cancelled (CANCELLED = 1) and diverted (DIVERTED = 1) flights, as their arrival delay values are not meaningful for on-time performance modelling. Third, records with null values in FL_DATE, DEP_TIME, ARR_DELAY, or DISTANCE were dropped.

We then computed the following columns:

- `day_of_week`, `year`, `month`, and `dep_hour`: extracted from FL_DATE
- `distance_group`: four-bucket variable that takes DISTANCE into 500, 1,000, and 2,000 miles
- `is_late`: **Layer 1 target** which separates on-time flights (arrival delay ≤ 15 minutes, label 0) from late flights (> 15 minutes, label 1)
- `delay_severity`: **Layer 2 target** defined only over late flights and separates minor delays (arrival delay ≤ 45 minutes, label 0) from major delays (> 45 minutes, label 1)

The final cleaned dataset comprises 3,480,548 flight records across 17 features, covering arrivals at ATL, ORD, and JFK between 2018 and 2022.

3.2 Exploratory Data Analysis

EDA was conducted in Apache Spark with visualizations produced via Matplotlib and Seaborn.

Figure 1 shows the arrival delay distribution, which exhibits a strong right skew: the majority of flights arrive on time or early. This motivates a hierarchical binary classification approach rather than a single multiclass formulation. Layer 1 separates on-time flights (arrival delay ≤ 15 minutes) from late flights (> 15 minutes), yielding a class distribution of 84.3% and 15.7%, respectively. This class imbalance is addressed through class weighting during Layer 1 training. For Layer 2, the severity threshold is set at 45 minutes, which corresponds closely to the median delay among late flights, as seen in Figure 2. This produces a near-balanced split of 51.4% minor (15–45 minute delay) and 48.6% major (> 45 minute delay) across 547,511 late flights; therefore, no resampling is required at Layer 2.

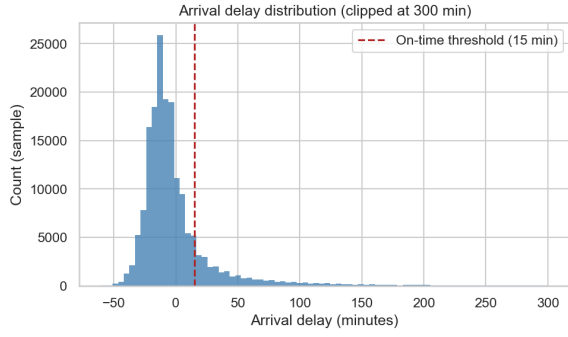


Figure 1: A histogram of flight arrival times.

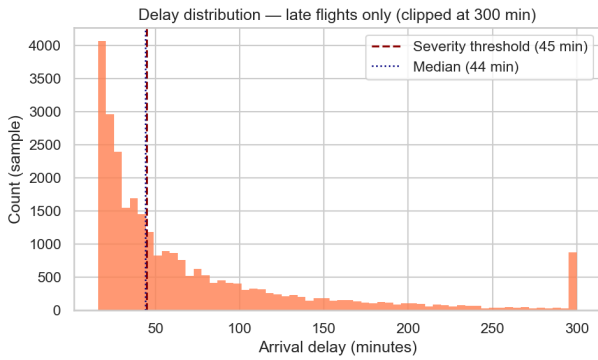


Figure 2: A histogram of delayed flight arrivals.

Figure 3 shows that late flight rates exhibit clear intra-day structure, with rates lowest in the early morning, reflecting clean schedules at the start of the operating day. They rise steadily through the afternoon and peak in the late evening, which is a well-documented consequence of delay propagation as aircraft accumulate schedule deviations across successive rotations. Overnight departures show the highest rates of all, exceeding 65% at 2:00; however, these flights represent a small fraction of total volume and are driven by flights already significantly disrupted. Seasonally, June and July exhibit the highest late rates at approximately 20%, consistent with increased traffic volumes due to summer holidays. February and September are the lowest, at roughly 12–13%.

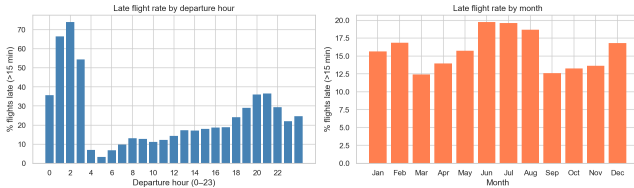


Figure 3: A temporal distribution of flights by hour of day (left) and month of year (right).

Wednesday and Thursday show the lowest late rates (~ 14%), while Friday and Saturday are the highest (~ 17%). The overall

spread across days is modest at roughly 3 percentage points, suggesting day of week is a weaker predictor than hour of departure or month.

The three hub airports differ substantially in both volume and delay behavior, as seen in Figure 4. ATL handles the largest share of traffic at approximately 1.65 million flights but has the lowest late rate at 12.8%. JFK handles the fewest flights at 516,338 but has the highest late rate at 20.1%, which is consistent with its constrained airspace and high traffic density. ORD falls between the two at approximately 1.32 million flights and a 17.7% late rate. These differences justify retaining destination airport as a model feature rather than training separate per-airport models.

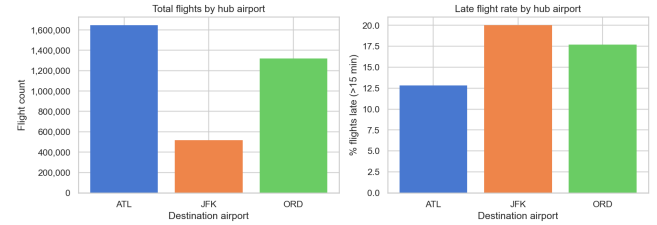


Figure 4: Flight makeup by hub: volume and late rate.

Late rates vary substantially across the 18 carriers in the dataset. Hawaiian Airlines has the highest rate at 30.4%, though its sample is very small (1,433 flights), making this figure unreliable. Among carriers with meaningful sample sizes, Delta (26.4%, 33,202 flights) and Frontier (25.7%, 196,692 flights) show the highest rates. United has the lowest late rate at 11.8% and by far the largest volume at 1,185,733 flights, which is over three times the next largest carrier. The spread of roughly 18 percentage points across carriers confirms carrier identity as an informative feature.

Among late flights, NAS congestion is the most frequent recorded cause (301,213 flights), followed closely by late arriving aircraft (260,779) and carrier issues (260,749). Weather accounts for approximately 35,000 flights and security delays are negligible. These five binary indicators are excluded from the model feature set as they are only observable post-departure; their inclusion would constitute data leakage. Late rates increase modestly with distance, from 14.2% for short-haul flights (≤ 500 miles) to 17.7% for long-haul ($\leq 2,000$ miles), with a slight dip to 17.1% for extra-long haul ($> 2,000$ miles). The dip likely reflects larger schedule buffers built into long-haul itineraries. Distance is retained as a feature given its modest but consistent signal.

3.3 Feature Engineering

3.3.1 Feature Selection. Several columns present in the cleaned dataset were excluded from the feature vector on principled grounds. The five binary delay cause indicators and `delay_none` are only observable post-departure and were excluded to prevent data leakage. `arr_delay` was excluded as the target variables are derived directly from it. `year` was excluded from the feature vector to avoid the model learning COVID-era traffic anomalies that would not generalise to future data, though it is retained as a partitioning variable for the train/test split. `distance_group` was excluded due

to near-perfect collinearity with distance ($r = 0.92$), which would violate the independence assumption of Logistic Regression.

3.3.2 Categorical Encoding. Three categorical features require encoding prior to model training. `carrier_id` (18 unique carriers) and `dest` (3 hub airports: ORD, JFK, ATL) are low-cardinality and were encoded using *StringIndexer* followed by *OneHotEncoder*, producing lossless sparse binary representations of 17 and 2 dimensions, respectively. `origin` presents a high-cardinality encoding challenge with approximately 241 unique departure airports. One-hot encoding would produce a 240-dimensional sparse vector, substantially increasing feature dimensionality and training time. Instead, *FeatureHasher* was applied with `numFeatures=64`, mapping origin codes to a fixed-size dense vector via a hashing trick. This reduces the origin’s contribution to the feature vector from 240 to 64 dimensions, at the cost of a small increase in the probability of hash collisions between airports. At this cardinality and hash size, collision probability is low, and the performance impact is expected to be negligible.

3.3.3 Feature Vector. The final feature vector was assembled using *VectorAssembler*, combining four numeric features (`dep_hour`, `day_of_week`, `month`, `distance`), the two OHE vectors, and the 64-dimension hashed origin vector, yielding a final dimensionality of 89 features. The assembled dataset was written to Parquet format and shared across all modelling notebooks to ensure consistent inputs across all models.

3.4 Model Training & Evaluation

3.4.1 Train/Test Split. A time-based split was applied across all models: flights from 2019 to 2021 were used for training, and 2022 was held out as the test set, yielding an approximate 80/20 split of 2,784,642 training rows and 695,906 test rows. A temporal split was preferred over random splitting to reflect the realistic deployment scenario in which a model trained on historical data is applied to future flights, and to avoid leaking future scheduling patterns into training.

3.4.2 Two-Layer Classification. Rather than a single multiclass formulation, we adopt a hierarchical binary classification approach motivated by the distribution of arrival delays. Layer 1 distinguishes on-time flights (arrival delay ≤ 15 minutes) from late flights (> 15 minutes) across all 3,480,548 flights. Layer 2 is trained exclusively on late flights and distinguishes minor delays (15-45 minutes) from major delays (> 45 minutes). The 45-minute threshold was selected empirically: it corresponds to the median delay among late flights (44 minutes), producing a near-balanced class split of 51.4% minor and 48.6% major across 547,511 late flights. Layer 2 training uses approximately 437,000 rows from the training partition.

3.4.3 Class Imbalance. Layer 1 exhibits a 5.4:1 class imbalance (84.3% on-time, 15.7% late). For models where class weighting is applicable (LR and RF), inverse-frequency class weights were computed from the training partition only using $w_c = \frac{N}{k \cdot N_c}$ and applying them via the `weightCol` parameter in Spark MLlib. Both weighted and unweighted variants are evaluated for LR and RF to show the effect of weighting on the evaluation metrics. Layer 2 requires no such adjustment, given its near-balanced distribution.

3.4.4 Evaluation Metrics. Given the operational context of making predictions intended to support airline resource planning, we prioritise precision over recall. False positives (predicting a delay that does not occur) incur direct costs through unnecessary crew reassignment, passenger rebooking, and fuel adjustments. Three primary metrics are reported consistently across all models:

- **AUC-ROC:** measures overall discriminative ability independent of classification threshold.
- **Weighted Precision:** the proportion of delay predictions that are correct, weighted by class frequency.
- **F0.5 Score:** the weighted harmonic mean of precision and recall with $\beta = 0.5$, placing twice the weight on precision. Defined as $F_{0.5} = \frac{1.25 \cdot \text{precision} \cdot \text{recall}}{0.25 \cdot \text{precision} + \text{recall}}$.

Weighted recall is additionally reported for Logistic Regression to contextualise the precision-recall tradeoff as a point of discussion.

3.5 Model Architecture

3.5.1 Logistic Regression. Logistic Regression (LR) serves as the interpretable baseline for both classification layers. Models were trained using Spark MLlib’s LR function with `maxIter=100` and default regularisation parameters of `regParam` and `elasticNetParam` set to 0.0. Prior to training, pairwise correlations among retained numeric features confirmed no residual multicollinearity (maximum $|r| < 0.07$) following the removal of `distance_group`. Both target classes were verified to be present across all quartiles of departure hour, ruling out complete separation. With 2,784,642 training rows and 89 features, the samples-to-features ratio greatly exceeds the recommended minimum of 10:1, satisfying the large-sample requirement for reliable coefficient estimation.

3.5.2 Random Forest. Random Forest was selected as a strong baseline ensemble method for several reasons specific to this task. First, its bagging structure makes it naturally robust to overfitting on high-dimensional feature spaces such as our 89-dimensional vector containing both numeric and one-hot encoded categorical features. Second, unlike GBT, the Spark MLlib implementation of Random Forest natively supports a `weightCol` parameter, allowing inverse-frequency class weights to be applied directly during training without requiring dataset resampling. Class weights were computed using the balanced weighting formula $w_c = \frac{N}{k \cdot N_c}$, where N is the total number of training instances, k is the number of classes, and N_c is the count of class c . This upweights the minority delayed class relative to the majority on-time class, penalising misclassifications of delayed flights more heavily during tree splitting without altering the training set size. This makes it a more principled approach to handling the Layer 1 class imbalance than downsampling, as all training instances are retained. Third, the independence of each tree in the ensemble makes Random Forest highly parallelisable across Spark worker nodes, making it well suited to distributed training at scale. The feature vector consists of 89 features: four numeric features (`dep_hour`, `day_of_week`, `month`, `distance`), 17 one-hot encoded carrier columns, a 64-dimension hashed origin vector, 2 one-hot encoded destination columns, and additional binary delay indicator columns, all assembled via a Spark ML Pipeline for reproducibility and to ensure identical preprocessing is applied at both training and inference time. The year feature

was excluded to avoid overfitting to COVID-era disruption patterns that would not generalise to the 2022 test period. $\text{maxBins}=300$ was set to accommodate the high cardinality of the origin feature, which requires more bins than the Spark default to avoid information loss during tree splitting. Hyperparameter tuning was performed using CrossValidator with 2-fold cross-validation on a 30% stratified random sample of the training data, evaluating four combinations of $\text{numTrees} \in \{50, 100\}$ and $\text{maxDepth} \in \{5, 10\}$ using AUC-ROC as the selection criterion. Tuning on a sample rather than the full training partition was a deliberate scalability trade-off, fitting eight full Random Forest models across 2.78 million rows would be prohibitively expensive on a two-worker-node cluster. The search space was intentionally kept small for the same reason, focusing on the two parameters with the greatest impact on model complexity and generalisation. The best parameters were subsequently used to train final models on the full training partition for both layers. Two variants were trained for Layer 1 to investigate the effect of class weighting on the 5.4:1 class imbalance between on-time and delayed flights. The unweighted variant assigns equal importance to all training instances, while the weighted variant uses inverse-frequency class weights via `weightCol`, upweighting the minority delayed class. Layer 2 operates on the subset of flights predicted as delayed by Layer 1 and exhibits a more balanced class distribution, making class weighting unnecessary at this stage. This two-stage design is consistent across all four classifiers evaluated in this study, ensuring comparability of results across models.

3.5.3 Gradient Boosted Trees. Cross-validation was not performed for hyperparameter selection as GBT’s sequential tree-building makes it computationally expensive at the scale of our data. Parameters were therefore set to defaults appropriate for the dataset. $\text{maxIter}=50$ which provides sufficient boosting rounds to learn non-linear patterns without overfitting to the training data. $\text{maxDepth}=5$ was chosen since it is shallow enough to generalise without risk of overfitting, but deep enough to capture multi-way feature interactions, such as departure hour, month, and carrier combinations. $\text{stepSize}=0.1$ is a standard learning rate that balances convergence speed and generalisation. A lower value would improve AUC marginally but requires more trees to compensate, increasing training time further.

Since `GBTClassifier` does not support a `weightCol` like Logistic Regression and Random Forest, the class imbalance in Layer 1 was addressed through downsampling on-time flights to match the delayed class size, producing a balanced training set of approximately 781k rows. Downsampling was selected as the most practical alternative within Spark MLlib, there’s no native oversampling in MLlib for GBT, and implementing a custom loss function was outside the scope of this study.

3.5.4 Stacking Ensemble. A stacking ensemble is trained using Logistic Regression and Random Forest as base learners, with a Logistic Regression meta-model. We kept the same two-layer classification structure.

Five-fold cross-validation is used to generate the meta-training matrix rather than simply applying the base models to the training set directly [12]. If LR and RF were fitted on the full training partition and their predictions on those same rows used as meta-features, the base models would have already seen every training instance

during fitting, producing artificially well-calibrated probabilities that do not reflect true out-of-sample behaviour. The meta-model would then learn to weight predictions that are unrealistically confident, and this would not generalise to the test set. By generating each row’s meta-feature from a fold on which the base model was not trained, the OOF protocol ensures the meta-training matrix contains honestly calibrated predictions throughout. Five folds was chosen as a practical balance between calibration quality [4], each base model is trained on 80% of the available data per fold, and computational cost, given that each fold requires fitting both LR and RF on approximately 2.2 million rows.

For Layer 1, both base models target whether or not a flight is late and incorporate inverse-frequency class weights to address the 5.4:1 class imbalance. The LR base model uses $\text{maxIter} = 100$ and default regularisation. The RF base model uses 50 trees, a maximum depth of 10, $\text{maxBins} = 300$ to accommodate the hashed origin feature, and $\text{seed} = 42$ for reproducibility. Layer 2 base models target the severity of the delay on the just flights that were marked as late with no class weighting, with all other hyperparameters identical to Layer 1.

The meta-model receives a two-dimensional feature vector of LR and RF positive-class probabilities, and fits a Logistic Regression meta-learner. For Layer 1 the decision threshold is lowered to 0.35 to increase sensitivity to late-flight class and Layer 2 retains the default threshold of 0.5.

4 Numerical Results

Table 1 shows the numerical results of all of the models in their various forms.

Model	Accuracy	Precision	F0.5	AUC-ROC	Train Time
Log. Reg. ^{u1}	0.8199	0.8524	0.8457	0.6500	13.0s
Log. Reg. ^{w1}	0.6162	0.7679	0.7318	0.6525	7.7s
Log. Reg. ²	0.5688	0.5681	0.5683	0.5899	3.9s
RF ^{u1}	0.8199	0.6723	0.6974	0.6699	226.9s
RF ^{w1}	0.6346	0.7775	0.7440	0.6775	231.7s
RF ²	0.5948	0.5989	0.5981	0.6251	17.7s
GBT ¹	0.5953	0.7857	0.7160	0.7032	83.0s
GBT ²	0.6049	0.6060	0.6028	0.6558	52.9s
Stacking ¹	0.8203	0.7701	0.7796	0.6786	657.3s
Stacking ²	0.5929	0.5931	0.5931	0.6237	132.6s

Table 1: Summary of model evaluation results. Key: ¹ = Layer 1; ² = Layer 2; ^u = Unweighted; ^w = Weighted.

4.1 Logistic Regression

Logistic Regression (LR) serves as the interpretable baseline for both classification layers. Models were trained using Spark MLlib’s `LogisticRegression` with $\text{maxIter}=100$ and default regularisation parameters.

4.1.1 Layer 1 (On-time vs Late). Two variants were evaluated: unweighted and class-weighted, with the latter applying inverse-frequency weights to penalise misclassification of the minority late-flight class. The unweighted model achieves high accuracy (0.820) but is entirely degenerate. The confusion matrix shows it

predicts on-time for almost all 695,906 test flights (570,597 true negatives, just 4 true positives), yielding an F0.5 of 0.846, driven entirely by majority-class precision. This confirms the well-known failure mode of accuracy as a metric under class imbalance. The weighted model trades accuracy (0.616) for meaningful coverage of both classes: it correctly identifies 77,033 late flights against 48,276 false negatives, with an F0.5 of 0.732 and AUC-ROC of 0.653. The improvement in AUC from 0.650 to 0.653 is marginal, suggesting that while class weighting corrects the prediction bias, LR’s linear decision boundary limits its discriminative power on this problem.

The numeric feature coefficients reveal that departure hour dominates with a coefficient of 0.09, consistent with the EDA finding that late-evening flights have substantially higher delay rates. Day of week and month have much smaller positive effects, while distance has almost no effect, confirming that temporal scheduling carries more signal than route features for LR.

4.1.2 Layer 2 (Minor vs. Major Delay). Trained on late flights only, the Layer 2 model achieves an accuracy of 0.569, F0.5 of 0.568, and AUC-ROC of 0.590, which is barely above the random (0.5) baseline. The confusion matrix shows the model splitting predictions roughly evenly between classes, with 40,497 correct minor predictions and 30,775 correct major predictions, but substantial misclassification in both directions. This near-random performance is expected: distinguishing minor from major delays requires capturing complex non-linear interactions between features that LR cannot represent. These results motivate the ensemble approaches evaluated in subsequent sections.

4.2 Random Forest

Random Forest (RF) serves as the bagging-based ensemble baseline for both classification layers, complementing the boosting approach of GBT and the linear baseline of LR. Models were trained using Spark MLlib’s `RandomForestClassifier` with hyperparameters selected via cross-validation as described in Section 3.

4.2.1 Layer 1 (On-time vs Late). Two variants were evaluated: unweighted and class-weighted, with the latter applying inverse-frequency weights via `weightCol` to penalise misclassification of the minority late-flight class.

The unweighted model achieves an accuracy of 0.8199, matching the degenerate LR baseline. The confusion matrix confirms the same failure mode: the model overwhelmingly predicts on-time, producing a weighted F0.5 of 0.6974 driven by majority-class precision rather than genuine discriminative ability. The weighted F1 of 0.7388 and AUC-ROC of 0.6699 are marginally above LR (0.6500), suggesting that RF’s non-linear decision boundary captures slightly more signal from the feature space than LR’s linear classifier, but the improvement is modest.

The weighted variant trades accuracy (0.6346) for more balanced class coverage, achieving a weighted precision of 0.7775, F0.5 of 0.7440, and AUC-ROC of 0.6775. Compared to the weighted LR (F0.5 = 0.7318, AUC = 0.6525), the weighted RF improves across all metrics, consistent with its ability to model non-linear interactions between features such as departure hour, carrier, and origin combinations that LR cannot capture. Training times were 226.9s and 231.7s for the unweighted and weighted variants respectively,

substantially longer than LR (13.0s) due to the cost of constructing 100 trees across distributed partitions.

Feature importance analysis reveals that `dep_hour` is the dominant predictor (0.5405), consistent with the EDA finding that late-evening departures carry substantially higher delay rates. `carrier_id` contributes meaningfully as an aggregated OHE group (0.1985), suggesting that airline-specific operational factors are a stronger signal than previously captured by LR coefficients. `origin` contributes 0.1116, reflecting airport-level congestion effects, while temporal features `month` (0.0305) and `day_of_week` (0.0053) play a smaller role. Distance and distance group contribute minimally (0.0232 and 0.0134), consistent with the LR finding that route length is a weak predictor of delay onset.

4.2.2 Layer 2 (Minor vs Major Delay). The Layer 2 RF model achieves an accuracy of 0.5948, weighted F1 of 0.5851, F0.5 of 0.5981, and AUC-ROC of 0.6251. While marginally above LR (AUC = 0.5899), performance remains close to the random baseline (0.5), reflecting the inherent difficulty of distinguishing minor from major delays given the overlapping feature distributions near the 45-minute threshold. No class weighting was applied at this stage, as Layer 2 operates on the subset of flights already predicted as delayed, which exhibits a more balanced class distribution.

Feature importance at Layer 2 shifts relative to Layer 1. `dep_hour` remains dominant (0.4489), but distance rises to a more prominent role (0.0608 vs 0.0232 in Layer 1), suggesting that route length is a stronger driver of delay severity than delay onset, consistent with the GBT Layer 2 findings. `carrier_id` (0.1585) and `origin` (0.1372) remain important, while `month` (0.1083) increases in relative importance compared to Layer 1, suggesting that seasonal patterns have a stronger influence on how severe a delay becomes once a flight is already late. Training took 17.7s, considerably faster than Layer 1 due to the smaller training set.

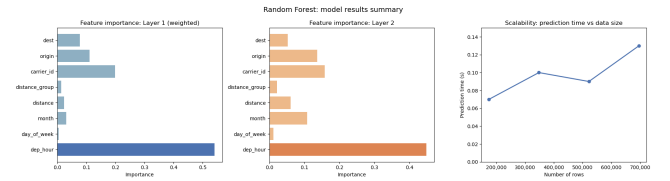


Figure 5: Layer 2 RF: Feature importance and scalability runtime for partitions of the dataset.

4.2.3 Scalability. Prediction time remains effectively constant across all partition sizes, ranging from 0.07s at 25% (174,185 rows) to 0.13s at 100% (695,906 rows), demonstrating strong inference scalability on the Dataproc cluster. The near-constant prediction time reflects Spark’s ability to distribute the scoring of a fitted forest across partitions in parallel, with the fixed coordination overhead dominating over the marginal cost of additional rows. Training times of 226.9s and 231.7s for the two Layer 1 variants are substantially longer than GBT (83.0s) and LR (13.0s), attributable to the cost of constructing 100 independent trees in parallel across 4 partitions on a two-worker-node cluster, compared to GBT’s sequential boosting on a downsampled dataset and LR’s single-pass gradient updates.

4.3 Gradient Boosted Trees

In contrast to Random Forest, Gradient Boosted Trees (GBT) builds an ensemble sequentially, where each tree corrects the residual errors of the previous one, directly minimising classification loss during each boosting round. Models were trained using Spark MLlib's GBTClassifier.

4.3.1 Layer 1 (On-time vs Late). GBT achieves a weighted precision of 0.7857, F0.5 of 0.7160, and AUC-ROC of 0.7032. The confusion matrix shows 325,224 true negatives and 89,059 true positives, but 245,373 false positives, so the model flags a significant number of on-time flights as delayed, which is a direct consequence of downsampling at the cost of on-time data loss. This is also reflected by the low accuracy (0.595). In feature importance, `dep_hour` (0.37), `month` (0.19) and `distance` (0.13) collectively dominate, with carrier identity contributing meaningfully through OHE columns ranking above `day_of_week`. Training took 83.0s on our Dataproc cluster, considerably longer than LR but faster than RF due to the reduced 781k training set from downsampling.

4.3.2 Layer 2 (Minor vs. Major Delay). Compared to Layer 1, performance drops across all metrics: AUC-ROC of 0.656, weighted precision of 0.606, and F0.5 of 0.603, reflecting the harder classification boundary between minor and major delays. The confusion matrix shows 19,386 false positives and 30,124 false negatives, suggesting that the model is more likely to misclassify major delays as minor than reverse and defaults to the minor class. This reflects the 45-minute threshold, where flights near the mark may be genuinely ambiguous regardless of model complexity. Feature importance shifts notably from Layer 1. While `dep_hour` remains dominant (0.39), `distance` rises to second (0.21), suggesting flight length is a stronger driver of delay severity than onset. `month` drops in relative importance, suggesting seasonality matters more for whether a flight is delayed than how badly. Training took 52.9s, faster than Layer 1 given the smaller training set (422k vs 781k rows).

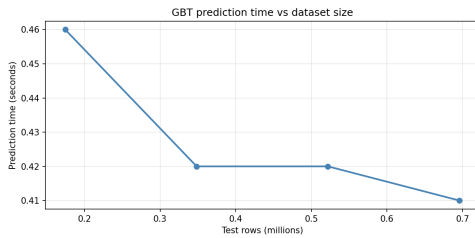


Figure 6: GBT Layer 1: Runtime for Partitions of Dataset

4.3.3 Scalability. As seen in 6, on Layer 1, GBT prediction time decreases from 0.46s to 0.41s as data size increases from 174k to 695k rows. As the dataset grows, Spark's fixed task setup and coordination costs are shared across more rows within each partition, reducing the cost per row. GBT predicts all data sizes in under 0.5s in this scalability experiment run, confirming it scales well for batch inference.

4.4 Stacking Ensemble

The stacking ensemble combines LR and RF base learners via a Logistic Regression meta-model trained on out-of-fold predictions from a 5-fold cross-validation protocol.

4.4.1 Layer 1 (On-time vs Late). The stacking ensemble achieves an accuracy of 0.820, a weighted precision of 0.770, an F0.5 of 0.780, and an AUC-ROC of 0.679. The confusion matrix reveals a highly conservative classifier. Of the 125,309 late flights in the test set, only 8,070 are correctly identified (TP), against 117,239 missed (FN), while 7,810 on-time flights are incorrectly flagged (FP) and 562,787 are correctly passed through (TN). This yields a minority-class precision of 0.508 and recall of 0.064.

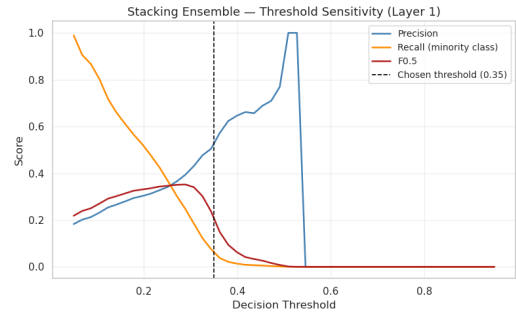


Figure 7: Threshold sensitivity of the stacking ensemble at Layer 1.

We also investigated whether the threshold of 0.35 was appropriately chosen given the results, Figure 7 plots precision, recall, and F0.5 across the full threshold range. We can see that F0.5 peaks at exactly 0.35 before collapsing sharply, confirming the 0.35 is optimal under the precision-first goal we are looking for in this study. This peak F0.5 of approximately 0.36 on the delayed class remains low regardless of threshold choice, which indicates the ceiling is not a consequence of threshold configuration but of the base learners limitations. The meta-model inherits the shortcomings of the LR and RF models, and no threshold adjustment can recover the recall lost upstream. The spike in precision near threshold 0.55 reflects near-zero positive predictions at that point and is not a valid operating decision. The consequence is that the stacking ensemble's high F0.5 (0.780) is driven largely by its strong on-time precision rather than meaningful any meaningful delay detection.

4.4.2 Layer 2 (Minor vs. Major Delay). The Layer 2 stack achieves an accuracy of 0.593, a weighted precision of 0.593, an F0.5 of 0.593, and an AUC-ROC of 0.624, with a training time of 132.6s. The confusion matrix shows a more balanced prediction profile, with 30,552 major delays being correctly identified (TP) and 43,744 minor delays being correctly classified (TN), against 30,191 false negatives and 20,822 false positives, with a minority-class precision of 0.595 and recall of 0.503. We can see that across both layers, there is a significant increase in computational power over the other models. The OOF design choice, which is necessary for preventing the meta-model seeing overfit base-learner predictions, is the direct explanation for stacking's training time, and that tradeoff isn't justified by the marginal metric gains at either layer.

4.5 Discussion

In Layer 1, GBT achieved the highest AUC-ROC of 0.703 and precision of 0.786 among rebalanced models, demonstrating it most reliably separates delayed from on-time flights and produces the fewest unnecessary delay predictions, directly reducing crew reassignment and rebooking costs. Stacking achieved the highest F0.5 of 0.780, best balancing true delay detection against false positives, making it the most operationally relevant model for airline resource planning where precision is prioritised. The unweighted LR and RF models achieve the highest accuracy (0.8199) and recall, but these are inflated by the 85/15 class imbalance since a model predicting on-time for nearly all flights scores well simply by taking advantage of the majority class. Among rebalanced models, Stacking achieves the highest meaningful accuracy of 0.8203, correctly classifying the greatest proportion of flights across both classes. In terms of training time, LR weighted is the fastest at 7.7s compared to 83.0s for GBT, 231.7s for RF, and 657.3s for Stacking, presenting a significant advantage for real-time or frequently retrained airline delay systems. Thus no single model dominates across all metrics. However, given the operational cost of false delay predictions for industry use, Stacking's F0.5 and GBT's AUC and precision advantages make them the strongest models for deployment at Layer 1.

In Layer 2, all model performance is weaker in comparison to Layer 1, reflecting the more difficult classification boundary between minor and major delays at the 45-minute threshold. Performance is broadly similar across GBT, RF, and Stacking, with AUC-ROC ranging narrowly from 0.624 to 0.656 and precision and F0.5 within 1-2 points across all three. GBT marginally leads on AUC (0.656) and accuracy (0.605). RF leads on precision (0.599) and F0.5 (0.598). Stacking closely follows both. LR performs weakest across all metrics with an AUC of 0.590, confirming its linear boundary is ill-equipped for severity classification. In terms of training efficiency, LR is the fastest at 3.9s, followed by RF at 17.7s, GBT at 52.9s, and Stacking at 132.6s. The marginal performance differences between GBT, RF, and Stacking do not justify the additional computational cost of GBT and Stacking, as RF delivers comparable results in a fraction of the time, making it the most practical Layer 2 deployment choice. The narrow gap across all models suggests the 45-minute severity threshold matters more than model choice on performance metrics.

5 Conclusion

Our study presented a two-stage distributed binary classification design for predicting whether a flight is delayed and the severity of that delay, built on Apache Spark MLlib and evaluated across three major US hub airports, JFK, ATL, and ORD, on over 3.4 million flight records. Four different classifiers were evaluated across both layers: Logistic Regression, Random Forest, Gradient Boosted Trees, and a Stacking ensemble combining LR and RF.

The chosen two-level binary classification proved valuable at decomposing the complexity of predicting delay into two explicit binary decisions. Layer 1 revealed class imbalance as a methodological consideration in flight data, as evidenced by the misleadingly high accuracy and recall metrics of unweighted models, while weighted and downsampled models exhibit a tradeoff between meaningful accuracy and precision. Examination of performance metrics and

execution time in Layer 1 positioned GBT and Stacking as the strongest choices for practical use in industry predictions of flight delay, since they minimised misclassification of on-time flights as delayed, reducing the unnecessary operational costs that less precise models would incur at the expense of faster training. Layer 2 presented a fundamentally different modelling problem than Layer 1. Unlike Layer 1 where model choice meaningfully differentiated performance, the marginal differences between models in Layer 2 demonstrated that capturing classification at the 45-minute threshold is difficult regardless of model complexity. Therefore, training time became a heavier consideration, making RF the most practical Layer 2 model given its comparable metrics to other top performers. Notably, distance replaced departure hour as the second strongest predictor in Layer 2, suggesting that flight length drives delay severity more than delay onset. Scalability analysis in Layer 1 across 4 incremental jumps in training data size further revealed RF to have near-constant prediction time, while LR scaled approximately linearly, and GBT decreased training time due to Spark's distributed task overhead.

This study has limitations worth considering for future work. First, the feature set does not include explicit weather features, though month and origin city implicitly capture some seasonal and regional weather patterns. Incorporating real-time weather features at both origin and destination airports may carry useful signal to improve predictions. Second, origin and destination are encoded as two separate features, which may carry more predictive signal if encoded as a single origin-destination pairing to capture flight route-level delay patterns, since routes may be systematically more delay-prone than others. Third, design choices in this study present opportunities for future refinement. The training data spans 2018 to 2022, including the COVID-19 period where flight volumes and delay patterns were unusual, influencing the learned model distributions. Finally, the 45-minute severity threshold was chosen empirically, but future work could explore whether data-driven approaches to selecting a boundary that produces more separable classes exists.

References

- [1] ACI World. 2025. Global air travel forecasted to reach 9.8 billion passengers in 2025, nearing the historic 10 billion milestone. <https://aci.aero>. Accessed: 2026-04-28.
- [2] S.A.A. AlBassam and D.N. AlShahrani. 2025. Flight delay prediction: Evaluating machine learning algorithms for enhanced accuracy. *PLOS One* 20, 12 (2025), e0335141. doi:10.1371/journal.pone.0335141
- [3] Anthropic. 2025. Claude (Version claude-sonnet-4-5). <https://www.claude.ai>. Large language model. Accessed: 2026.
- [4] Leo Breiman. 1996. Stacked Regressions. *Machine Learning* 24, 1 (1996), 49–64.
- [5] Bureau of Transportation Statistics. 2024. On-Time Performance — Reporting Carrier On-Time Performance (1987–present). https://www.transtats.bts.gov/DL_SelectFields.aspx?gnoyr_VQ=FGJ&QO_fu146_anzr=b0-gvzr. Data retrieved for years 2018–2022..
- [6] Federal Aviation Administration. 2026. 14 CFR 244.1 – Definitions. <https://www.ecfr.gov/current/title-14/part-244/section-244.1>. Accessed: 2026-04-28.
- [7] Grammarly Inc. 2025. Grammarly. <https://www.grammarly.com>. Grammar and spell-checking tool. Accessed: 2026.
- [8] I. Hatipoğlu and Ö. Tosun. 2024. Predictive Modeling of Flight Delays at an Airport Using Machine Learning Methods. *Applied Sciences* 14, 13 (2024), 5472. doi:10.3390/app14135472
- [9] Perplexity AI. 2025. Perplexity AI. <https://www.perplexity.ai>. AI-powered search engine. Accessed: 2026.
- [10] Balasubramanian Thiagarajan, Lakshminarasimhan Srinivasan, Aditya Vikram Sharma, Dinesh Sreekanthan, and Vineeth Vijayaraghavan. 2017. A Machine Learning Approach for Prediction of On-time Performance of Flights. In *2017 IEEE/AIAA 36th Digital Avionics Systems Conference (DASC)*.
- [11] Gerasimos Vonitsanos, Theodor Panagiotakopoulos, Andreas Kanavos, and Athanasios Tsakalidis. 2021. Forecasting Air Flight Delays and Enabling Smart Airport Services in Apache Spark. In *17th IFIP International Conference on Artificial Intelligence Applications and Innovations (AIAI)*. 407–417.
- [12] David H. Wolpert. 1992. Stacked Generalization. *Neural Networks* 5, 2 (1992), 241–259.
- [13] World Travel & Tourism Council. 2025. Travel & Tourism Economic Impact Research (EIR). <https://wttc.org/research/economic-impact>. Accessed: 2026-04-28.

A Additional Results

Statement of Individual Contributions

The contributions to the project by each individual can be found at the following [link](#).

Statement on Use of Generative AI

We acknowledge the use of generative AI tools Claude [3] and Perplexity AI [9], for code debugging, the assisting with visualisation code, and as a writing aid for structuring and refining prose in this report. It was also used to assist with literature search. Grammarly [7] was used for spelling and grammar checking throughout.